

State-Level Provenance over Upsert Sources

Implemented and wired into the demo as `root README §3.6 — make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true`. Off by default, **CP only** (§5.0 explains why CCAF cannot run the statement verbatim). Code: `StateProvenancePTF`, `StateVersion`, `09_state_provenance_sinks.fql`, `85_state_provenance.fql`.

Table of Contents

- **1.0 What "state-level" actually changes**
 - **2.0 The model: content-addressed versions**
 - **2.1 Version identity**
 - **2.2 Parents travel with the version, not beside it**
 - **2.3 Edges are a projection, not a source of truth**
 - **3.0 Why the four failure modes disappear**
 - **4.0 Implementation**
 - **4.1 Read the log, own the state**
 - **4.2 The PTF**
 - **4.3 The SQL**
 - **4.4 Sinks**
 - **4.5 Running it**
 - **5.0 CP vs CCAF: where the divergence actually lands**
 - **6.0 Limits, and what to verify first**
-

1.0 What "state-level" actually changes

`flink-collector.md §3.1` draws the line this document starts from: an isotope records an **itinerary** — one identity, ordered hops, a *path* — and that is truthful only while every step is 1:1. State-level lineage is never 1:1. A row's current value is derived from every change that ever touched it, which is a **DAG over versions**, not a path over messages.

So the first design decision is to stop overloading the isotope. In state mode, **ISOTOPE_APPEND_HOP is not used at all**. Asking "is a `-U/+U` pair one hop or two?" has no good answer because the question is wrong: a revision is not a movement. The hop list stays what it is — a message-level artifact riding the log — and state provenance becomes a second, parallel record keyed by *version* rather than by trace. The two coexist without either lying.

The four problems that make the windowed merge collector unworkable on upsert sources — no window to key on, two statements drifting apart, changelog-blind projections, and time-zone-dependent bounds — are all symptoms of running a path model over a changelog. Change the model and they stop being problems rather than getting patched.

2.0 The model: content-addressed versions

2.1 Version identity

Every emitted state of a row gets a `version_id` derived from its own content:

```
digest      = SHA-256( source_name || SEP || entity_key || SEP || content )
version_id  = event_time_ms (48 bits) || digest[0:10]  → patched to a
valid UUIDv7
```

`SEP` is the same NUL field separator `MergeTrace` already uses, with the same null marker discipline. The high 48 bits carry event time exactly as `Isotope.uuidV7Bytes` and `MergeTrace` lay them out, so version IDs sort chronologically and interleave with interceptor-minted trace IDs rather than being 16 opaque hashed bytes.

That makes identity **(event time, content)**, not content alone — worth stating plainly. Two byte-identical states at the same event time are the same version, which is the idempotence that actually matters under at-least-once delivery; a row that revisits an earlier value *later* is a distinct version, which is what a version chain should say. The preimage is a wire contract: `StateVersionTest` pins field order, separator forgery, null-vs-empty, tombstones, and the UUIDv7 layout, so changing any of them is a build failure rather than silently renumbered history.

Content addressing buys four properties that matter more here than they did for the windowed merge:

- **No window required.** Identity comes from the value, so unbounded `GROUP BY`, regular joins, and dedup all have something to key on — the thing the window was standing in for.
- **Deterministic across runtimes and across reprocessing.** CP and CCAF derive the same ID from the same content; so does a replay six months later.
- **Idempotent under duplicate emission.** A retract stream revises the same logical row repeatedly, and at-least-once delivery repeats records. Recomputing a version you already published yields the same ID, so the provenance topic tolerates duplicates by construction.
- **Retraction becomes append.** You never mutate a version; you publish a new one. The provenance stream is therefore **append-only even though its subject is an updating table** — which is what lets every operator in the pipeline stay append-only.

A row that reverts to a previous value produces a previously seen ID. That is a deliberate property: identical content is the same state. If they need to distinguish "reverted to X" from "was X," carry `prev_version_id` on the record — the chain disambiguates, the identity doesn't have to.

2.2 Parents travel with the version, not beside it

Each provenance record carries its parent set **inline**:

Column	Carries
<code>version_id</code>	This state's content-addressed identity
<code>entity_key</code>	The upsert PK this version belongs to
<code>source_name</code>	The table this state was observed on — part of the preimage, so the same payload on two tables is two versions
<code>op</code>	<code>UPSERT</code> / <code>DELETE</code> , from the change that produced it

Column	Carries
parents	ARRAY<STRING> of the versions this one was derived from. In a 1:1 stage that is the version it supersedes, so the chain is <code>parents[0]</code> — no separate <code>prev_version_id</code> column, because a redundant one is a column that can disagree
parent_overflow	Parents dropped past the inline cap, 0 normally
emitted_at	Event time of the change

One record, one operator, one state. The merged output and its parent set can no longer disagree, because they are not two things.

2.3 Edges are a projection, not a source of truth

If a flat edge table is wanted for joins, derive it *downstream* of the provenance topic:

```
INSERT INTO state_provenance_edges
SELECT `version_id`, `entity_key`, `parent`, `emitted_at`
FROM state_provenance
CROSS JOIN UNNEST(`parents`) AS t(`parent`);
```

This is a pure projection of an already-consistent record, over an append-only topic. It cannot drift from what it flattens — unlike `81_merge_edge_markers.fql`, which is a second statement racing the first over the same source.

3.0 Why the four failure modes disappear

Objection to the windowed design	Why it does not apply here
Unbounded <code>GROUP BY</code> / regular join / dedup have nothing to key on	Identity is derived from content, not from window bounds. No window is needed for any of them.
Two statements, two watermarks, <code>event_count 1000</code> vs <code>edge_list 1001</code>	There is one statement and one operator. The parent set is a column of the record it describes; drift is not expressible.
The <code>isotope</code> view is changelog-blind	Nothing consumes a changelog. The pipeline reads the physical log append-only and reconstructs state inside a PTF (§4.1), so no operator ever sees an update.
<code>UNIX_TIMESTAMP(CAST(window_start AS STRING))</code> is time-zone-dependent	No window bounds appear in any identity. Event time is carried as epoch millis end to end, never round-tripped through a formatted string.

4.0 Implementation

4.1 Read the log, own the state

The move that makes everything else work: **do not consume the upsert view — consume the log underneath it, in append mode, and rebuild state yourself inside the PTF.**

Flink's restrictions on updating streams (window TVFs refuse them, time attributes do not survive them, plain sinks reject them) all vanish, because nothing in the pipeline is an updating stream. A compacted topic's messages are already an append-only sequence of changes; a Debezium topic is even more explicitly so. The isotope's headers are present on every one of those physical messages regardless of how any other consumer chooses to interpret the topic — including on tombstones, which means **deletes get first-class provenance**, something a state view cannot give you.

The cost is that you are now responsible for the state machine that `upsert-kafka` would have run for you. That is the trade: you take on state management and get changelog-free planning, deterministic identity, and consistent multi-output emission in exchange.

4.2 The PTF

`StateProvenancePTF` follows the idiom `StuckTracePTF` established — a `@StateHint` POJO, a `SET_SEMANTIC_TABLE + REQUIRE_ON_TIME` input, `ctx.timeContext(Instant.class)` for event time. Per entity it holds the current version ID and the parents folded in since the last publication; per input row it derives the new version, suppresses the emission entirely if it equals the current one (a redelivery is not a new state), and collects one row.

Three constraints worth knowing before writing it, all learned the hard way on this project:

- **CCAF rejects `MapView/ListView` in PTF state.** A plain `Map/List` field is the documented replacement — and it *still* rejects a `byte[]` map **value**, where a Base64 `String` works. Both fail at `CREATE FUNCTION` time, not at runtime, so a local CP test will not catch them.
- **Emit one row type.** `cp-flink 2.1.2`'s Expander round-trip rejects `CREATE VIEW` over a PTF, which is why the existing PTFs are invoked from `INSERT INTO` directly. Emitting state rows and edge rows as one tagged stream to be split by a view is therefore not available — another reason the parent set is a column rather than a second output.
- **Bound the parent set.** `PARENT_CAP` plus `parent_overflow` keeps the record and the state bounded. Entity-scoped derivations sit far below any sane cap; a global aggregate will blow through it, and for that case carry a count and a sketch rather than the members.

4.3 The SQL

One statement per derived table, on both runtimes:

```
INSERT INTO `isotope_state_provenance`
SELECT `version_id`, `entity_key`, `source_name`, `op`, `parents`,
`parent_overflow`, `emitted_at`
FROM TABLE(
  STATE_PROVENANCE(
    input    => TABLE `entity_log` PARTITION BY `entity_key`,
    on_time  => DESCRIPTOR(`event_time`),
    uid      => 'state-provenance-v1'
```

```
)
);
```

`entity_log (09_state_provenance_sinks.fql)` is the append-mode change log over the order topics, which are already declared `value.format = 'raw'` with headers as `VIRTUAL` columns — indifferent to value format, and tolerant of tombstones.

The demo's entity key. A real deployment keys on the upsert PK. This demo has none to offer:

`App.java` sets the record key to the producing service name and `hop` forwards the `DemoEvent` value verbatim, so neither identifies an entity. The isotope trace ID does — stamped once at the origin, surviving every hop — so the demo treats one traced order as the entity and its three stage records as three versions of it. Because `source_name` is in the preimage, those are three distinct versions even though the payload bytes never change:

```
orders.placed → orders.enriched → orders.fulfilled
```

each naming its predecessor in `parents`. That is the same shape a real upsert entity produces.

4.4 Sinks

- `isotope_state_provenance` — append-only. Plain `kafka` sink on CP (`avro-confluent`), append mode on CCAF. This is the durable lineage record.
- `state_provenance_edges` — append-only, the §2.3 `UNNEST` projection. Not created by the demo; derive it if you want flat edges.
- `current_state` — the only upsert table in the design, and it is a *convenience view* for "what is order 42 now," keyed by `entity_key`. Nothing in the provenance path reads it, so its changelog mode infects nothing.

4.5 Running it

```
make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true

# both collectors – they answer different questions, and neither implies
# the other
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
ENABLE_STATE_PROVENANCE=true
```

Off, nothing is created: no view, no topic, no statement, and the deployment is byte-identical to a build without the feature. Teardown removes `isotope_state_provenance` whether or not the flag was passed, so a `down` run cannot strand a topic an earlier `up` created.

5.0 CP vs CCAF: where the divergence actually lands

The feature ships CP-only, and the reason is narrow: canonicalization, not capability.

The version preimage needs bytes that are stable for a given state. CP's source tables are declared `'value.format' = 'raw'`, so the raw value is available as a `BYTES` column and the PTF hashes it directly. CCAF's Topic Catalog imports each topic with typed Protobuf columns and does not hand back the raw value, so the identical statement cannot be written there — it would hash a canonical rendering of the typed columns instead. That is a perfectly good design, and arguably a better one, but it yields different IDs for identical data, so shipping both would mean two incompatible identity schemes behind one feature flag. There is deliberately no `ENABLE_STATE_PROVENANCE` on `make cc-flink-reports-up`.

Everything else is portable. `StateProvenancePTF`'s state is plain `String/List` fields precisely so CCAF would accept it, and the shadow JAR CCAF already uploads contains the class — it is simply never registered there. **No capability divergence:** the design avoids the one place the runtimes genuinely differ — CCAF exposes `changelog.mode = retract` and stock open-source Flink has no retract-capable Kafka sink — by never needing retract semantics at all. Content-addressed immutable versions turn what would have been a retract stream into an append stream.

To close the gap, pick one canonicalization both runtimes can compute — typed columns rendered by a shared UDF is the obvious candidate — and the CCAF side becomes the same three statements the merge feature already has there.

What remains is the declaration-layer asymmetry this project already documents for headers in [flink-collector.md §2.2](#):

	CP / open-source Flink	CCAF
Source table in append mode	own the DDL: <code>'connector' = 'kafka'</code>	Topic Catalog auto-imports; set <code>changelog.mode</code> via <code>ALTER TABLE</code>
Provenance sink	<code>kafka + avro-confluent</code>	append mode + Protobuf/SR, as the seven reports already differ
<code>current_state</code> table	<code>upsert-kafka</code> + PK	<code>changelog.mode = upsert + PRIMARY KEY ... NOT ENFORCED</code>
The PTF	same shadow JAR, registered programmatically	same shadow JAR, <code>CREATE FUNCTION ... USING JAR</code>
PTF state	<code>MapView/ListView</code> allowed	plain <code>Map/List</code> only; no <code>byte[]</code> map values

The last row is a coding constraint, not a behavioral difference: write the state POJO to CCAF's rules and the identical class runs on both. That is the same discipline the existing two PTFs already follow.

6.0 Limits, and what to verify first

Verify before building:

1. **Flipping `changelog.mode` on a shared CCAF catalog table** affects every consumer of that table. If that is not acceptable, a second append-mode table over the same topic is needed — confirm CCAF permits that shape in their environment.

2. **Writable `headers` metadata on an upsert-mode sink**, if they ever want the collector writing to one. Not needed for this design; needed if message-level tracing is layered on top.
3. **Canonicalization stability** across the language boundary if anything outside Flink computes a `version_id`. Two implementations of "canonical content" is two chances to disagree.

Accepted limits:

- **Ancestry is queryable, not traversable.** Flink SQL has no recursive CTEs, so walking more than one generation belongs in a relational or graph store fed by the provenance topic — the same boundary [flink-collector.md §3.1](#) already sets for merge edges.
- **Compaction bounds replay.** Tracing the physical log means superseded messages eventually vanish. Live provenance is unaffected; historical reconstruction is bounded by the source topic's compaction policy, not its retention. The provenance topic itself should not be compacted.
- **Global aggregates do not fit.** The inline parent set is right for entity-scoped derivation and wrong for a `SUM` over everything. That case wants a count plus a sketch, and loses the ability to name the parents.
- **Idle partitions stall event time.** Upsert topics are frequently bursty and partially idle; the 5s watermark delay tuned for the demo is not a starting point for their volumes.